HW

SUMMARY NOTES

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([1.5, 2.5, 3.5])

print(a.dtype)
print(b.dtype)

c = a + b
print(c)
print(c.dtype)
```

```
int64
float64
[2.5 4.5 6.5]
float64
```

```
import numpy as np

arr = np.array([10.5, 20.7, 30.9], dtype=int)

print(arr)
print(arr.dtype)

result = arr + 5
print(result)

print(result * 2)
```

```
[10 20 30]
int64
[15 25 35]
[30 50 70]
```

```
arr = np.array(["AJ", "Raj", "Riya"], dtype=int)

print(arr)
print(arr.dtype)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_7809/1350358144.py in <cell line: 0>()
----> 1 arr = np.array(["AJ", "Raj", "Riya"], dtype=int)
      2
      3 print(arr)
      4 print(arr.dtype)

ValueError: invalid literal for int() with base 10: 'AJ'
```

Next steps: Explain error

```
import numpy as np

arr = np.array([0, 1, 2, 0, 5], dtype=bool)

print(arr)

result = arr * 1
print(result)

result = result + 5
print(result)

print(result.dtype)
```

```
[False  True  True False  True]
[0 1 1 0 1]
[5 6 6 5 6]
int64
```

```
import numpy as np

arr = np.zeros((3, 4))

print(arr)
print(arr + 10)
print(arr * 5)
print(arr + 100)
```

```
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
[[10. 10. 10. 10.]
 [10. 10. 10. 10.]
 [10. 10. 10. 10.]]
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
[[100. 100. 100. 100.]
 [100. 100. 100. 100.]
 [100. 100. 100. 100.]]
```

Challenge Quetions - DPP

```
import numpy as np

arr = np.ones((2, 3))
print(arr)

result = arr * 10
print(result)

result = result + 5
print(result)

result = result / 5
print(result)
```

```
[[1. 1. 1.]
 [1. 1. 1.]]
[[10. 10. 10.]
 [10. 10. 10.]]
[[15. 15. 15.]
 [15. 15. 15.]]
[[3. 3. 3.]
 [3. 3. 3.]]
```

```
import numpy as np

arr = np.full(4, 50)
print(arr)

arr = arr + 20
print(arr)

arr = arr * 2
print(arr)

arr = arr - 10
print(arr)
```

```
[50 50 50 50]
[70 70 70 70]
[140 140 140 140]
[130 130 130 130]
```

```
import numpy as np

celsius = np.array([0, 10, 20, 30, 40])

f = celsius*9/5 + 32

print(celsius)
print(f)
```

```
[ 0 10 20 30 40]
[ 32.  50.  68.  86. 104.]
```

Case-study Questions

```
# A student has scored:

# 65, 72, 80, 88, 91

# Create a NumPy array containing these marks.
# Add 5 bonus marks to every subject and display the original marks and updated marks.
```

```
marks = np.array([65,72,80,88,91])
bonus = 5
updated_marks = marks + bonus
print(marks)
print(updated_marks)
```

```
[65 72 80 88 91]
[70 77 85 93 96]
```

```
# A shop has five products with prices:

# 100, 250, 400, 550, 800

# Create a NumPy array
# and calculate the price after applying a 10% discount to every product.
```

```
prices=np.array([100,250,400,550,800])
discount=(prices*10)/100
prices_after_discount=prices - discount
print("Original Prices ",prices)
print("Prices after Discount is ",prices_after_discount)
```

```
Original Prices  [100 250 400 550 800]
Prices after Discount is  [ 90. 225. 360. 495. 720.]
```

```
prices=np.array([100,250,400,550,800])

discount = prices*0.1
prices_after_discount = prices*0.9

print("Original Prices ",prices)
print("Prices after Discount is ",prices_after_discount)
```

```
Original Prices  [100 250 400 550 800]
Prices after Discount is  [ 90. 225. 360. 495. 720.]
```

```
# A company has five employees with salaries:

# 30000, 35000, 40000, 45000, 50000

# Create an array and give every employee a 10% salary increment.
```

```
salary=np.array([30000,35000,40000,45000,50000])
increment=salary*0.1
print("Original Salary ",salary)
print("Salary After Increment ",salary+increment)
```

```
Original Salary  [30000 35000 40000 45000 50000]
Salary After Increment  [33000. 38500. 44000. 49500. 55000.]
```

```
salary=np.array([30000,35000,40000,45000,50000])
print("Original Salary ",salary)
print("Salary After Increment ",salary*1.1)
```

```
Original Salary  [30000 35000 40000 45000 50000]
Salary After Increment  [33000. 38500. 44000. 49500. 55000.]
```

```
A student's marks are stored in a tuple:

(78, 85, 92, 67, 88)

Convert the tuple into a NumPy array and add 3 marks to every value.
```

```
a = (78, 85, 92, 67, 88)

np_a = np.array(a)
print(np_a+3)
```

```
[81 88 95 70 91]
```

```
# A company records sales for four quarters:

# Q1 = 125000
# Q2 = 148000
# Q3 = 135000
# Q4 = 172000

# Create a NumPy array.

# The company expects next year's sales to be 15% higher than this year's sales.
#Calculate the expected sales for each quarter.

# Then calculate the additional sales expected in each quarter.
```

```
import numpy as np
sales=np.array([125000,148000,135000,172000])
new_sales=sales*1.15
additional_sales=sales*0.15
print(new_sales)
print(additional_sales)
```

```
[143750. 170200. 155250. 197800.]
[18750. 22200. 20250. 25800.]
```

```
# Two branches of a company have inventory for five products.

# Branch A = [120, 150, 180, 100, 200]
# Branch B = [80, 130, 170, 120, 150]

# Create two NumPy arrays.

# Calculate:

# Total inventory for each product.
# Difference between Branch A and Branch B.
# Combined inventory after receiving an additional 20 units of every product.
```

```
A = np.array([120, 150, 180, 100, 200])
B = np.array([80, 130, 170, 120, 150])

print("Total inventory:", A + B)
print("Difference:", abs(A - B))
print("Additional 20 units:", A + B + 20)
```

```
A store sells four products.

Prices:

[250, 400, 750, 1200]

Units sold:

[12, 8, 5, 10]

Calculate the revenue generated by each product.

The store then wants to increase the revenue figures by 5% for forecasting purposes.
```

```
import numpy as np
```

```
prices = np.array([250, 400, 750, 1200])
units = np.array([12, 8, 5, 10])

revenue = prices * units
forecast = revenue * 1.05

print("Revenue:", revenue)
print("Forecast:", forecast)
```

```
Revenue: [ 3000  3200  3750 12000]
Forecast: [ 3150.   3360.   3937.5 12600. ]
```

```
import numpy as np

marks = np.array([70, 80, 65, 90, 85])

print(marks)
```

```
[70 80 65 90 85]
```

```
len(marks)
```

```
5
```

```
marks.shape
```

```
(5,)
```

```
arr2 = np.array([
    [80, 75, 90],
    [70, 85, 88],
    [92, 90, 95]
])

print(arr2)
```

```
[[80 75 90]
 [70 85 88]
 [92 90 95]]
```

```
marks.ndim
```

```
1
```

```
arr2.ndim
```

```
2
```

```
a = np.array([
    [
        [1, 2],
        [3, 4]
    ],
    [
        [5, 6],
        [7, 8]
    ]
])

print(a)
print(a.ndim)
```

```
[[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]
3
```

```
sales = np.array([
    [100, 120, 150,200],
    [130, 180, 160,150],
    [200, 220, 250, 170]
])
```

```
print(sales)
print(sales.ndim)
```

```
[[100 120 150 200]
 [130 180 160 150]
 [200 220 250 170]]
2
```

```
sales.shape
```

```
(3, 4)
```

```
sales.size
```

```
12
```

```
len(sales)
```

```
3
```

```
arr = np.array([10.5, 20.7, 30.9])

arr_int = arr.astype(int)

print(arr)
```

```
[10.5 20.7 30.9]
```

```
print(arr_int)
```

```
[10 20 30]
```

```
arr = np.array([10.5, 20.7, 30.9], dtype = int)
```

```
arr
```

```
array([10, 20, 30])
```

```
data = np.array([
    [10, 20, 30, 40],
    [50, 60, 70, 80],
    [90, 100, 110, 120]
])

print("len:", len(data))
print("size:", data.size)
print("shape:", data.shape)
print("ndim:", data.ndim)
```

```
len: 3
size: 12
shape: (3, 4)
ndim: 2
```

```
student_marks = np.array([
    [78, 85, 90, 88, 76],
    [92, 81, 85, 89, 95],
    [67, 72, 70, 75, 68],
    [88, 91, 94, 90, 87],
    [76, 80, 79, 82, 85]
])
```

```
# The array
# Number of dimensions
# Shape
# Total number of values
# Data type
# Number of students
# Number of subjects
```

```
student_marks = np.array([
    [78, 85, 90, 88, 76],
    [78, 85, 90, 88, 76],
    [78, 85, 90, 88, 76],
    [78, 85, 90, 88, 76]
])

print(student_marks)
print(student_marks.ndim)
print(student_marks.shape)
print(student_marks.size)
print(student_marks.dtype)
print(student_marks.shape[0])
print(student_marks.shape[1])
```

```
[[78 85 90 88 76]
 [78 85 90 88 76]
 [78 85 90 88 76]
 [78 85 90 88 76]]
2
(4, 5)
20
int64
4
5
```

A college wants to initialize a marks table for 5 students and 4 subjects.

Initially, every student's marks are set to zero.

Create this using NumPy.

Then create another 5 × 4 array containing a default value of 50.

```
student_marks = np.zeros((5, 4))
print(student_marks)

print(student_marks + 50)

# OR

updated = np.full((5, 4), 50)
print(updated)
```

```
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
[[50. 50. 50. 50.]
 [50. 50. 50. 50.]
 [50. 50. 50. 50.]
 [50. 50. 50. 50.]
 [50. 50. 50. 50.]]
[[50 50 50 50]
 [50 50 50 50]
 [50 50 50 50]
 [50 50 50 50]
 [50 50 50 50]]
```

Five students have scores:

[45, 56, 67, 78, 89]

The teacher gives everyone 3 grace marks.

After that, the college applies a 10% scaling factor to the adjusted scores.

Calculate the final scores.

```
import numpy as np
scores = np.array([45,56,67,78,89])
grace = scores + 3
final_scores = grace*1.1
print(grace)
print(final_scores)
```

```
[48 59 70 81 92]
[ 52.8  64.9  77.   89.1 101.2]
```

A company wants to set targets from ₹20,000 to ₹1,00,000.

Generate exactly 9 target values, equally spaced.

Then create actual sales:

[18000, 28000, 39000, 47000, 57000, 66000, 76000, 90000, 105000]

Calculate the difference between actual and target.

```
import numpy as np

targets = np.linspace(20000, 100000, 9)

actual_sales = np.array([
    18000, 28000, 39000, 47000,
    57000, 66000, 76000, 90000, 105000
])

difference = actual_sales - targets

print("Targets:", targets)
print("Actual:", actual_sales)
print("Difference:", difference)
```

```
Targets: [ 20000.  30000.  40000.  50000.  60000.  70000.  80000.  90000. 100000.]
Actual: [ 18000  28000  39000  47000  57000  66000  76000  90000 105000]
Difference: [-2000. -2000. -1000. -3000. -3000. -4000. -4000.     0.  5000.]
```

A dataset contains:

[10, 20, 30.5, 40, 50.75]

Create a NumPy array and display its data type.

Then create an integer version of the array.

Finally, multiply the integer array by 3.

```
import numpy as np

data = np.array([10, 20, 30.5, 40, 50.75])

print("Original:", data)
print("Original dtype:", data.dtype)

integer_data = data.astype(int)

result = integer_data * 3

print("Integer array:", integer_data)
print("Result:", result)
```

```
Original: [10.   20.   30.5  40.   50.75]
Original dtype: float64
Integer array: [10 20 30 40 50]
Result: [ 30  60  90 120 150]
```

A company has product prices:

[499, 799, 1299, 1999, 2499]

The company wants to:

Increase all prices by 12%.
Add ₹50 packaging charges.
Calculate the final customer price.

```
import numpy as np

prices = np.array([499, 799, 1299, 1999, 2499])
```

```
increase = prices * 0.12
revised_price = prices + increase

final_price = revised_price + 50

print("Original prices:", prices)
print("Revised prices:", revised_price)
print("Final prices:", final_price)
```

```
Original prices: [ 499  799 1299 1999 2499]
Revised prices: [ 558.88  894.88 1454.88 2238.88 2798.88]
Final prices: [ 608.88  944.88 1504.88 2288.88 2848.88]
```

A website offers:

First discount: 10%
Second discount: 5% on the already discounted price

Prices:

[1000, 2000, 3000, 4000]

Calculate the final prices.

A business has monthly revenue:

[80000, 95000, 110000, 125000, 140000, 155000]

The expected growth rate is 12%.

Calculate:

Expected increase.
Forecast revenue.
Create a target revenue array using linspace() from ₹80,000 to ₹1,60,000 with exactly 6 values.
Compare forecast revenue with the target.

A factory wants to generate production quantities starting from 100 units, increasing by 50 units each day, for 10 days.

Create the array using arange().

Then increase every production target by 10%.